python-ai-chatbot documentation

python-ai-chatbot is a Command-line Interface Socratic-tutor chatbot built on the Anthropic API, with Retrieval-Augmented Generation so it can answer questions using content from documents the user provides, and a Streamlit web User Interface

This page contains auto-generated API reference documentation using Sphinx, generated from the project's docstrings

The project's setup/usage instructions live in the README.md file

See the API reference page below

Skip to content

API Reference

A Command-line Interface Socratic-tutor chatbot using the Anthropic API

```
class project.ChatBot
```

Skip to content

The stateful core - Message history, client initialisation, retriever/document state, command specific behaviour

act_based_on_command(command: str, argument: str | None) → None

Matches each recognised command to a specific action

PARAMETERS:

- **command** – One of the existing commands
- **argument** – An argument if there is one or None if there isn't

hold_conversation() → None

The main loop: greets, reads user input and skips it if blank and checks for command use or routes to chat. Exits gracefully on /quit or Ctrl+C

reset_history() → None

Clears the conversation history

respond_with_instruction(command: str, instruction: str) → None

Shared helper for commands that need the LLM to do something by building an extended system prompt and sending it

PARAMETERS:

- **command** – One of the existing commands; sent as the message content to the LLM
- **instruction** – The instruction to be concatenated with the base system prompt to produce different behaviour

retrieve_context(message: str, system_prompt: str) → str

Used by the Streamlit UI (app.py), not the CLI. Returns system prompt extended with additional context retrieved from loaded document. Falls back to the given system prompt unchanged if no document is loaded or retrieval fails.

PARAMETERS:

- **message** – The user's current message; combined with recent history to form the retrieval query
- **system_prompt** – The base system prompt to augment

RETURNS:

The augmented system prompt if retrieval succeeds; the original system prompt otherwise

retrieve_prompt_with_context(message: str, system_prompt: str) → str

Returns system prompt extended with additional context retrieved from loaded document, printing the retrieved chunks first. Falls back to the given system prompt unchanged if no document is loaded or retrieval fails.

PARAMETERS:

- **message** – The user's current message; combined with recent history to form the retrieval query
- **system_prompt** – The base system prompt to augment

RETURNS:

The augmented system prompt if retrieval succeeds; the original system prompt otherwise

```
send_message(message: str, system_prompt: str = SYSTEM_PROMPT) → str
```

Sends message to LLM, first grounding the system prompt in the loaded document's content if one exists. On success, both the prompt and response get appended to history and the history is trimmed. On failure, it prints and returns an error message without touching the history.

PARAMETERS:

- **message** – The prompt from the user
- **system_prompt** – Optional argument. Used for configuring the chatbot's underlying behaviour and specifically making it act as a tutor

RETURNS:

The response from the chatbot on success; Error message on failure

```
stream_response(message: str, system_prompt: str = SYSTEM_PROMPT) →
    Iterator[str]
```

Used by the Streamlit UI (app.py), not the CLI. Sends message and system prompt to API, yielding response text chunks as they arrive. On success, appends the user message and full response to history and trims it. On failure, it raises an exception without touching history

PARAMETERS:

- **message** – The prompt from the user
- **system_prompt** – Optional argument. Used for configuring the chatbot's underlying behaviour

RAISES:

- **anthropic.APIConnectionError** – If any network connection failures occur
- **anthropic.AuthenticationError** – If the API key is invalid, expired or revoked
- **anthropic.RateLimitError** – If the rate limit is exceeded
- **anthropic.APIStatusError** – If any HTTP errors occur
- **anthropic.AnthropicError** – If anything other than the above goes wrong regarding the API
- **ValueError** – If the response is empty or has non-text content

YIELD:

Chunks of the response text as they arrive

```
project.build_retrieval_query(current_message: str, history: list[MessageParam])
    → str
```

Builds a string containing the user's current message and their last two exchanges with the assistant so that it has more context and returns it

PARAMETERS:

- **current_message** – The current prompt of the user
- **history** – The list containing message history

RETURNS:

String containing the previous two user-assistant exchanges and the user's current message

```
project.explain_llm_error(error: AnthropicError) → str
```

Returns a friendly error message for each caught anthropic exception

PARAMETERS:

error – The general AnthropicError caught

RETURNS:

[illegible] error message


```
project.get_llm_response(messages: list[MessageParam], client: Anthropic,
    system_prompt: str) → str
```

Sends message history and system prompt to API, streaming and printing the response text as it arrives. On success, returns the full response. On failure, it raises an exception

PARAMETERS:

- **messages** – The list containing message history
- **client** – The Anthropic client
- **system_prompt** – The string of text used to configure the chatbot's underlying behaviour

RAISES:

- **anthropic.APIConnectionError** – If any network connection failures occur
- **anthropic.AuthenticationError** – If the API key is invalid, expired or revoked
- **anthropic.RateLimitError** – If the rate limit is exceeded
- **anthropic.APIStatusError** – If any HTTP errors occur
- **anthropic.AnthropicError** – If anything other than the above goes wrong regarding the API
- **ValueError** – If the response is empty or has non-text content

RETURNS:

The response string provided it comes back fine

```
project.load_conversation(path: str) → list[MessageParam]
```

Reads a JSON file at a given path into the list containing message history

PARAMETERS:

path – The path at which the file to read from lives

RAISES:

- **FileNotFoundError** – If the file cannot be found
- **OSError** – If any other thing goes wrong when reading file
- **json.JSONDecodeError** – If the file is not a valid JSON document
- **UnicodeDecodeError** – If the file is not UTF-8 encoded

RETURNS:

The message history read from the file

```
project.main() → None
```

The entry point of the program - Constructs the ChatBot object and starts the conversation loop

```
project.parse_command(user_input: str) → tuple[str, str | None] | None
```

Skip to content

Extracts the command and argument (if there is one) and returns them. If the command doesn't exist or user input is empty, None is returned.

PARAMETERS:

user_input – The user prompt

RETURNS:

A tuple containing a command (if matched with one) and an argument (None if not provided). If unrecognised command or empty input, None is returned

```
project.save_conversation(messages: list[MessageParam], path: str) → None
```

Writes the message history list to a JSON file at a given path, creating the parent directories if needed

PARAMETERS:

- **messages** – The list containing message history
- **path** – The path at which to create and store the file

RAISES:

- **PermissionError** – If user does not have the necessary permissions to write to the given path
- **OSError** – If any other thing goes wrong when saving conversation

```
project.trim_history(messages: list[MessageParam], limit: int) →
    list[MessageParam]
```

Trims the history list to the most recent *limit* entries. Strips any leading assistant replies to preserve user-assistant pairs

PARAMETERS:

- **messages** – The list containing message history
- **limit** – The number of most recent messages to keep

RETURNS:

The trimmed list containing message history

Document loading, chunking, embedding and retrieval for the RAG feature using the Voyage AI API

```
class rag.Retriever
```

Skip to content

Holds an indexed document's chunks and embeddings, and retrieves the most relevant chunks for a given query

`embed_or_error(texts: list[str], input_type: str) → list[list[float]] | list[list[int]] | str`

Embeds the given texts and returns them on success. On failure, it returns an error message

PARAMETERS:

- **texts** – The list of texts to embed
- **input_type** – Either "query" or "document" per Voyage AI's embedding API

RETURNS:

The list of embeddings on success; friendly error message on failure

`index_document(path: str) → None | str`

Reads a text or PDF file from a given path, splits it into chunks, embeds them and stores the chunks/embeddings on success. On failure, it returns an error message

PARAMETERS:

path – The path from which to index

RETURNS:

None on success; friendly error message on failure

`rerank_or_error(query: str, candidates: list[str]) → list[str] | str`

Reranks the candidate documents by their relevance to the query and returns the top TOP_K on success. On failure, it returns an error message

PARAMETERS:

- **query** – The search query
- **candidates** – The list of candidate documents i.e. the documents retrieved by the search with embeddings

RETURNS:

The list of TOP_K candidate documents on success; friendly error message on failure

`retrieve(query: str) → list[str] | str`

Embeds the query, narrows the indexed chunks to the RERANK_CANDIDATES most similar by cosine similarity, then reranks them and returns the top TOP_K most relevant

PARAMETERS:

query – The text to find relevant chunks for

RETURNS:

A list of the most relevant chunks (up to TOP_K) on success; friendly error message on failure

```
rag.augment_system_prompt(system_prompt: str, chunks: list[str]) → str
```

Builds a system prompt with additional context by appending the retrieved file chunks and instructing the model to use them and say when the answer isn't contained in them

PARAMETERS:

- **system_prompt** – The base system prompt
- **chunks** – The list of retrieved chunks to ground the prompt in

RETURNS:

The system prompt extended with the context

```
rag.embed_texts(texts: list[str], client: Client, input_type: str) →
    list[list[float]] | list[list[int]]
```

Sends texts to the Voyage API and returns their embeddings

PARAMETERS:

- **texts** – The list of texts to embed (document chunks or a single query)
- **client** – The Voyage AI client
- **input_type** – Either "query" or "document" per Voyage AI's embedding API

RAISES:

- **voyageai.error.APIConnectionError** – If any network connection failures occur
- **voyageai.error.AuthenticationError** – If the API key is invalid, expired or revoked
- **voyageai.error.RateLimitError** – If the rate limit is exceeded
- **voyageai.error.VoyageError** – If anything other than the above goes wrong regarding the API

RETURNS:

The list of embeddings, one per input text

```
rag.explain_api_error(error: VoyageError) → str
```

Skip to content

Returns a friendly error message for each caught Voyage AI exception

PARAMETERS:

error – The general VoyageError caught

RETURNS:

A friendly error message

`rag.read_document_as_string(path: str) → str`

Reads a document's content as a string, detecting whether it's a PDF or a plain text file and acting appropriately

PARAMETERS:

path – The path from which to read the document

RAISES:

- **ValueError** – If the file is a PDF larger than MAX_PDF_SIZE_BYTES
- **FileNotFoundError** – If the file cannot be found
- **PermissionError** – If user does not have the necessary permissions to read from given path
- **UnicodeDecodeError** – If a plain text file is not UTF-8 encoded
- **pypdf.errors.FileNotDecryptedError** – If an encrypted PDF has not been successfully decrypted
- **pypdf.errors.PyPdfError** – If the PDF is corrupted, empty or unreadable
- **OSError** – If any other thing goes wrong when reading the file

RETURNS:

The string containing the document's content

`rag.read_file_as_string(path: str) → str`

Reads a file's content as a UTF-8 encoded string and returns a string containing content

PARAMETERS:

path – The path from which to read file

RAISES:

- **FileNotFoundError** – If the file cannot be found
- **PermissionError** – If user does not have the necessary permissions to read from given path
- **UnicodeDecodeError** – If the file is not UTF-8 encoded
- **OSError** – If any other thing goes wrong when reading file

RETURNS:

Skip to content g containing file content

`rag.read_pdf_as_string(path: str) → str`

Reads a PDF's content and returns a string containing it

PARAMETERS:

path – The path from which to read the PDF

RAISES:

- **FileNotFoundError** – If the file cannot be found
- **PermissionError** – If user does not have the necessary permissions to read from given path
- **pypdf.errors.FileNotDecryptedError** – If an encrypted PDF has not been successfully decrypted
- **pypdf.errors.PyPdfError** – If the PDF is corrupted, empty or unreadable
- **OSError** – If any other thing goes wrong when reading the file

RETURNS:

The string containing the PDF's content

`rag.split_text_into_chunks(text: str) → list[str]`

Splits text into CHUNK_SIZE_WORDS-sized chunks. Each chunk overlaps with the next by CHUNK_OVERLAP_WORDS words

PARAMETERS:

text – The text to split

RETURNS:

A list of the overlapping, fixed-size chunks

A Streamlit web UI for the Socratic study-assistant chatbot, reusing project.py's ChatBot class

`app.display_and_clear(key: str, display_function: Callable[[str], object]) → None`

Displays a pending message stored in session_state under the given key and then clears it if one exists

PARAMETERS:

- **key** – The session_state key that may hold a pending message
- **display_function** – The Streamlit function used to display the message (e.g. st.error, st.success)

`app.handle_conversation_upload() → None`

Used as a file_uploader callback for /load. Restores the uploaded conversation as the current history and stores the result message for display

Skip to content `ment_upload() → None`

Used as a file_uploader callback for /loaddoc. Indexes the newly uploaded document into the retriever and stores the result message for display; clears the retriever if the file is removed

`app.handle_instruction_button(command: str, instruction: str) → None`

Used as a button callback. Builds an extended system prompt from the instruction and gets the full response, storing an error message for display if the call fails.

PARAMETERS:

- **command** – One of the existing commands; sent as the message content to the LLM
- **instruction** – The instruction to be concatenated with the base system prompt to produce different behaviour

`app.handle_reset_click() → None`

Used as a button callback. Clears conversation history, forgets which files were already processed and clears the loaded document's retriever